

when there are portions that are going to be the same every time, such as the power circuit. Also, there are certain hardware circuits such as pushbuttons and LEDs that greatly aid in testing and debugging. Having all of the hardware and circuitry already assembled makes experimentation and general prototyping much easier.

If you build this basic setup yourself with a general-purpose PCB or breadboard, you can never be 100 per cent sure about the connections. Every time something does not work, you do not know whether it is because of the setup or the development you are carrying out. Thus a lot of time is wasted debugging the issue.

Also, if you plan to use SMD chips in the final product, building the basic setup becomes very time-consuming. You cannot build such a setup with general-purpose PCBs and have to design a dedicated PCB. A lot of time is wasted due to the lead times for manufacturing this PCB, sourcing components and assembly before you can really start with the development.

So it is better to use a suitable development board with ready interface and immediately start moving on with your project. These development boards can be used in multiple projects, so it is a one-time investment that will not hit your budget much.

Typically, components of a development board include:

1. Power circuit (typically, set up to run from a 9V/12V power supply)
2. Programming interface (to program the microcontroller through computer)
3. Basic input circuitry, like buttons
4. Basic output circuitry, like LEDs
5. Input/output (I/O) pins access (to be used for everything else such as motors, sensors and LCD screens)

Which development board to use?

Selection of the development board mainly depends on the microcontroller that you plan to use. You can also do it the other way around; select the development board first and then plan the

project based on that microcontroller.

Selecting the right microcontroller for your product application is itself a challenging task. There is a huge variety of microcontrollers available in the market. DigiKey, for instance, shows more than 15,000 different items under 'microcontroller' search, and it is almost impossible to find the one best suited for you if you have not figured out your requirement beforehand.

Some engineers jump to microcontroller selection even before the system overview has been worked out, which is definitely a bad idea. Engineers should first work out the high levels of the system, block diagram and flowchart and then select the microcontroller with the information available.

You can consider the following points to select the right microcontroller:

Hardware interfaces required. Using the block diagram that you made earlier, list out all the interfaces you need for external hardware. First, figure out the communication interfaces needed. This will narrow down your choices. Communication interfaces commonly inbuilt in microcontrollers are Ethernet, CAN, USB, serial ports, I2C, SPI and UART.

Next, determine the required digital inputs and outputs, analogue-to-digital converter inputs, PWM, etc. This will further narrow down choices for you.

Computing power. To decide on the computing power needed, analyse the architecture and requirements of the software that will be burnt into the microcontroller. Check whether the software has parts such as floating-point mathematics, high-frequency control loops that require more computing power. Also check the subtasks and how often they run. This will give you an idea of the processing power required, which is actually indicated by the clock speed of the microcontroller.

Within each CPU there is an electronic clock. The clock's job is to create a series of electrical pulses at regular intervals. This allows the computer to synchronise all its components, deter-

mining the speed at which it can pull data from its memory and perform calculations. When you talk about how many gigahertz or megahertz your processor has, you are really talking about clock speed.

Architecture. If you have worked out the hardware interface and computing power requirements correctly, you will automatically land up on selecting the architecture of the microcontroller for your project.

The architecture of a microcontroller refers to the philosophy of internal implementation. All architectures are well-proven but one might work better than the other for a particular application. Decide on the architecture based on the hardware interface and computing power requirements as well as the future requirements. Here consider whether an 8-bit core would be sufficient for future developments too, or you need a better core. In any case, the decision has to be made between 8-bit and 32-bit as 16-bit is not much popular these days.

Memory. Flash and RAM are very critical components of any microcontroller and it is important to make sure that you do not run out of program space or variable space. Using the software architecture and the communication peripherals included in the application, you can estimate how much Flash and RAM will be required for the application.

Power. You will now have a list of potential candidates that suit your requirement. This is the right time to examine the power requirements of the part. If the device will be powered from a battery, select a low-power microcontroller.

Cost. Do not forget to examine the price per piece and minimum-order quantities. Make a smart decision as the price will greatly affect the overall cost of the product.

Hardware and software tools. Finally, check the availability and quality of development tools. Minimally, you will need some sort of programmer to load the program into the microcontroller. Here availability of the devel-